Web Programming
CSE 3120

Open-Ended Lab Report

Retool Campus — A Peer-to-Peer Campus Tool & Equipment Library

Submitted By

Name: Yeamin Eram Chowdhury

ID: 242014209

Submitted To

Nasir Uddin Ahmed

Lecturer,

Department of Computer Science and Engineering (CSE)

University of Liberal Arts Bangladesh (ULAB)

Introduction

Retool Campus is a dynamic, database-driven web application built as the Open-Ended Lab project for my web programming course. This platform functions as a peer-to-peer tool and equipment library for the university community. Here the students and staff can list underused items such as electronics, cameras, and lab tools, and borrow items listed by others instead of purchasing new ones. By enabling item-sharing within the campus, the system aims to reduce electronic waste and unnecessary spending while strengthening a culture of resource circulation on campus.

The application follows a classic three-tier web architecture. The presentation layer is built with semantic HTML, CSS and a small amount of vanilla JavaScript for client-side interaction. The application/business logic layer is implemented in plain PHP 8 (no framework), organized into reusable includes for authentication, helper functions, and shared layout components. The data layer is a normalized MySQL/MariaDB database accessed exclusively through PDO with prepared statements. The system was developed and tested locally using a XAMPP-style stack, with the database created and managed through phpMyAdmin and the site can be viewed locally from my laptop at this link – localhost/campus-tool-library/.

This report documents the objectives, implementation details, results, and ethical/technical compliance considerations of the Campus Tool Workshop project, in line with the four task areas of the lab: application functionality, database design, codebase quality, and reporting.

Objectives

The project was built with the following objectives:

- To design and implement a full-stack, database-driven web application using HTML, CSS, JavaScript, PHP, and MySQL.
- To build a complete borrow/lend workflow — from browsing and listing items to requesting, approving, declining, and returning items — that is equivalent with a real-world peer-to-peer sharing service.
- To design a normalized relational database schema capable of representing users, categories, items, borrow requests, and the resulting social impact of completed loans.
- To ensure a secure and safe authentication, a session-based user authentication with proper password hashing, along with account and profile management features.
- To apply defensive coding practices throughout the codebase: parameterized SQL queries to prevent injection, CSRF tokens on every state-changing form, output escaping to prevent XSS, and ownership checks to enforce authorization.
- To evaluate the frontend against the Web Content Accessibility Guidelines (WCAG) 2.0 and document the extent to which the interface meets recognized accessibility success criteria.
- To produce a working, demonstrable prototype that reflects good software engineering practice within the scope of an academic open-ended lab.

Implementation

1. System Architecture

The application is organized as a set of top-level PHP pages (index.php, register.php, login.php, logout.php, add_item.php, item.php, dashboard.php, request_action.php, profile.php) that each handle both the display (GET) and the processing (POST) of a specific feature, following the common PHP pattern of self-processing pages. Shared logic — the database connection, session/authentication helpers, CSRF protection, and layout partials — lives in an includes/ directory (auth.php, functions.php, header.php, footer.php) and is pulled in with require_once at the top of every page, which keeps the codebase DRY and centralizes security-critical logic such as CSRF verification and login checks.

2. Authentication and Session Security

User registration (register.php) validates the submitted name, email format, student/staff ID, and password length/confirmation server-side before checking for a duplicate email and inserting the new user with a securely hashed password using PHP's password_hash(). Login (login.php) looks up the user by email and verifies the password with password_verify(), and on success calls session_regenerate_id(true) to defend against session fixation before storing the user ID in $_SESSION. logout.php clears and destroys the session entirely. Every page that requires an authenticated user calls require_login() (from includes/auth.php) as its first action, and every state-changing form embeds a CSRF token that is verified with verify_csrf() before any database write occurs.

3. Browsing, Search, and Filtering

index.php implements the public "pegboard" of listings. It dynamically builds a parameterized SQL query that filters by a free-text search term (matched against item title and description), a category ID, and an availability status, joining items with categories and users to display category icon, owner name, and live status badges. A small hero section also surfaces impact counters (items listed, loans completed, members) computed with simple aggregate queries.

4. Listing and Managing Items

add_item.php lets a logged-in user create a new listing. It validates title length, description length, category selection, and loan-length bounds, then handles an optional photo upload through handle_item_photo_upload() (MIME-type validated, size-limited, and randomly renamed to avoid path/overwrite attacks per the project's security notes). A unique, human-readable asset_code is generated and checked for collisions before the item is inserted. item.php shows full item detail, and allows the owner to delete their own listing (with an ownership check: DELETE ... WHERE id = ? AND owner_id = ?) or lets any other logged-in user submit a borrow request with start/end dates validated against today's date and the item's configured max_loan_days.

5. Borrow-Request Workflow

The borrowing lifecycle is coordinated between item.php (request creation), dashboard.php (viewing incoming and outgoing requests across three tabs — Listings, Incoming, Outgoing), and request_action.php (approve, decline, mark returned, or cancel a request). request_action.php wraps every transition in a database transaction: approving a request marks the item as borrowed and automatically rejects any other pending requests on the same item; marking a request as returned frees the item, logs the impact (estimated money saved) into impact_log, and rewards both parties with reputation points. Every transition is guarded by an is_owner / is_borrower check derived from the session, so a user can only act on requests that concern them.

6. Profile Management

profile.php allows a logged-in user to update their personal details (name, department, phone) and to change their password, with the current password re-verified via password_verify() before a new hash is stored. It also displays simple reputation statistics (items listed, loans completed, reputation points earned) pulled with COUNT() queries scoped to the current user.

7. Database Design

The schema is normalized around five tables — users, categories, items, borrow_requests, and impact_log — connected through foreign keys (items.owner_id → users.id, items.category_id → categories.id, borrow_requests.item_id → items.id, borrow_requests.borrower_id → users.id, impact_log.request_id → borrow_requests.id). This design avoids data duplication (e.g. category names are stored once and referenced by ID) and lets the impact_log table cleanly capture the environmental/financial benefit of each completed loan without polluting the borrow_requests table.

Table	Key Columns
users	id, full_name, email, student_id, password_hash, department, phone, avatar_initials, reputation_pts
categories	id, name, icon
items	id, owner_id (FK→users), category_id (FK→categories), title, description, item_condition, photo_path, max_loan_days, deposit_note, asset_code, status, created_at
borrow_requests	id, item_id (FK→items), borrower_id (FK→users), start_date, end_date, message, status, created_at
impact_log	id, request_id (FK→borrow_requests), est_money_saved, created_at

8. Security and Code Quality Practices

- Parameterized queries throughout — every SQL statement observed in the codebase (index.php, item.php, dashboard.php, add_item.php, request_action.php, profile.php) uses PDO prepared statements with bound parameters, eliminating SQL injection risk from string concatenation.
- CSRF protection — every POST form includes a hidden csrf_token field generated by csrf_token() and checked with verify_csrf() before the request is processed.
- Output escaping — all dynamic output is passed through an e() helper (a wrapper around htmlspecialchars()) before being echoed into HTML, mitigating stored/reflected XSS.
- Authorization / ownership checks — actions such as deleting a listing, approving or rejecting a request, and cancelling a request all re-verify that the acting user actually owns the relevant item or request before making any change.
- Transactional integrity — request_action.php wraps multi-statement state transitions in $pdo->beginTransaction() / commit() / rollBack() so an item's status and its request history can never fall out of sync.
- Client-side UX, server-side truth — HTML5 attributes (required, min, minlength, type="date") and small JS confirmations (confirm("Cancel this request?")) improve the user experience, but every rule is re-validated in PHP, since client-side checks alone are not trustworthy.

Results

The completed application was tested locally against the XAMPP stack described in the Setup section of the project README, with the MySQL schema imported via phpMyAdmin and the site served at localhost/campus-tool-library/. The following core user flows were exercised and confirmed to work as intended:

- Account registration and login, including validation error messages for weak passwords, mismatched confirmation, and duplicate emails.
- Browsing, searching, and filtering the pegboard by keyword, category, and availability status.
- Listing a new item with an optional photo upload, condition, category, and maximum loan length.
- Submitting, approving, declining, and cancelling borrow requests, and marking a completed loan as returned.
- Dashboard tabs correctly separating a user's own listings from incoming and outgoing borrow requests.
- Profile updates and password changes, with re-verification of the current password.

Screenshots of each of the above flows, captured from the local deployment, are inserted below to demonstrate the working system.

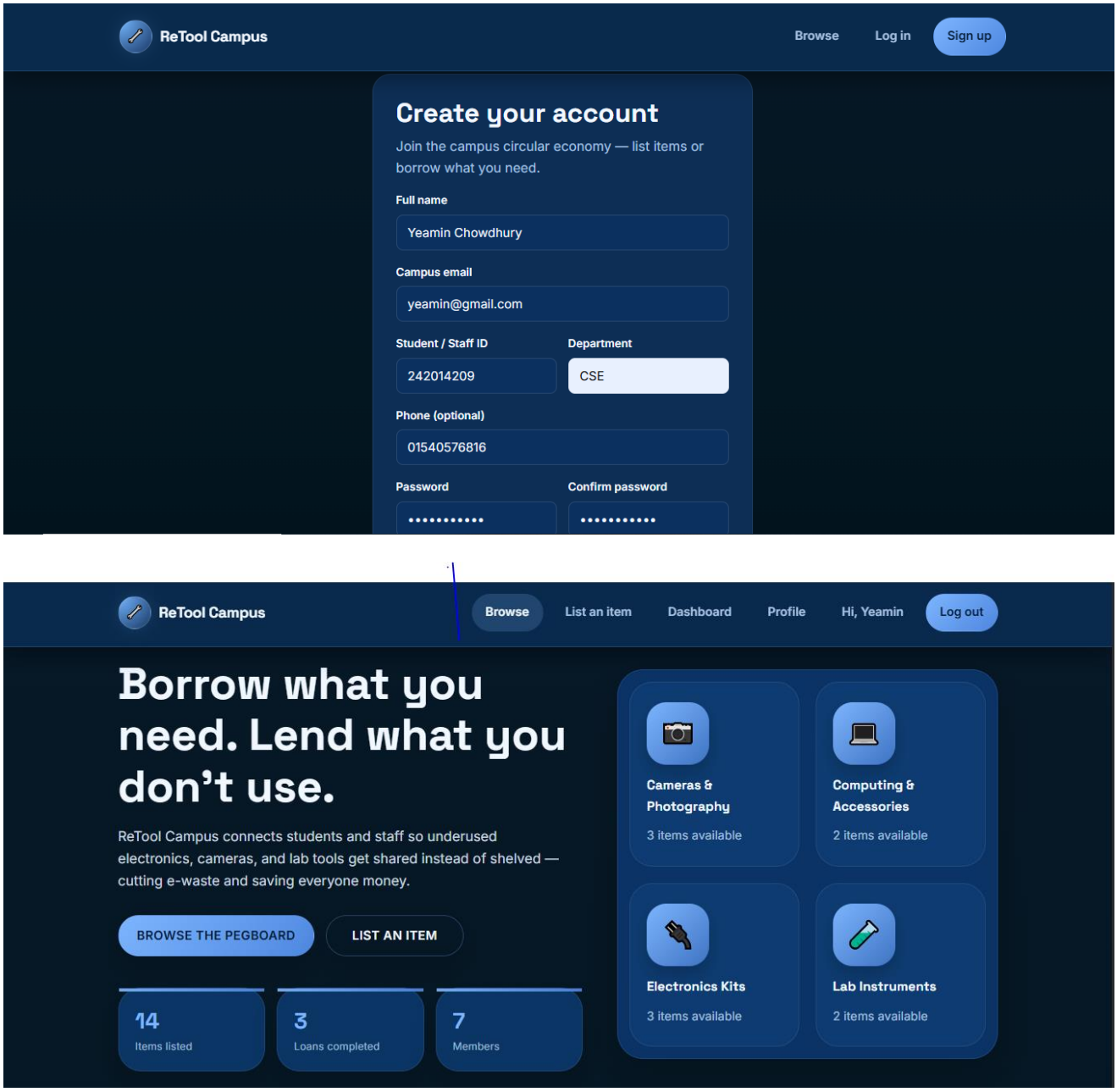


Fig 1 : Account registration and login, including validation error messages for weak passwords, mismatched confirmation, and duplicate emails.

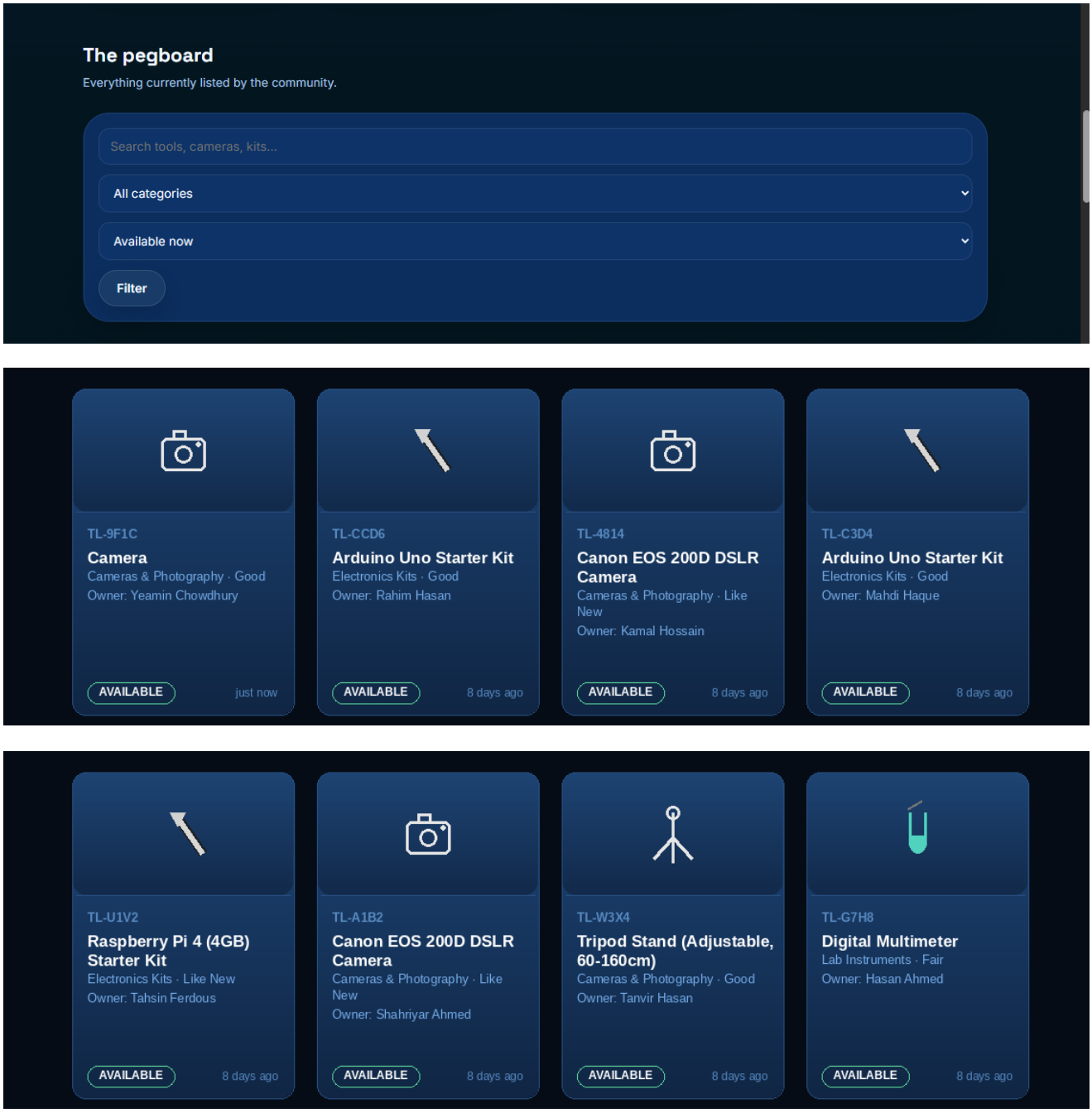


Fig 2 : Browsing, searching, and filtering the pegboard by keyword, category, and availability status.

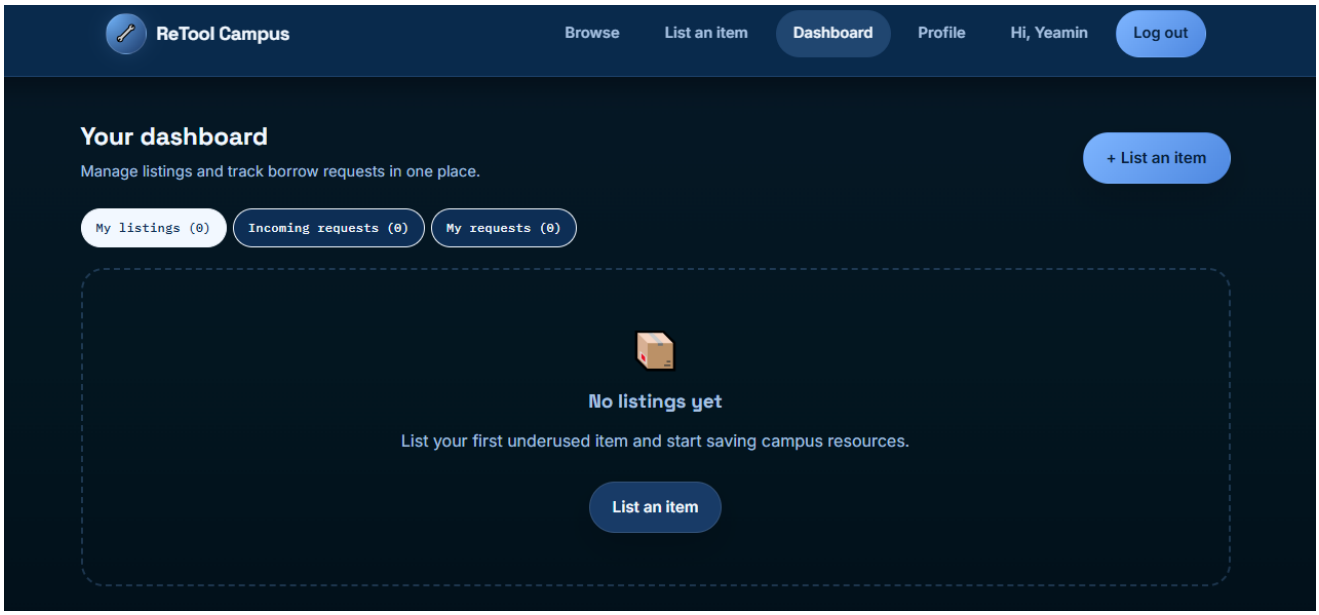


Fig 3 : User dashboard where the user can have the option to list items and can see the incoming requests and their own requests. The items listed will be displayed in the list here.

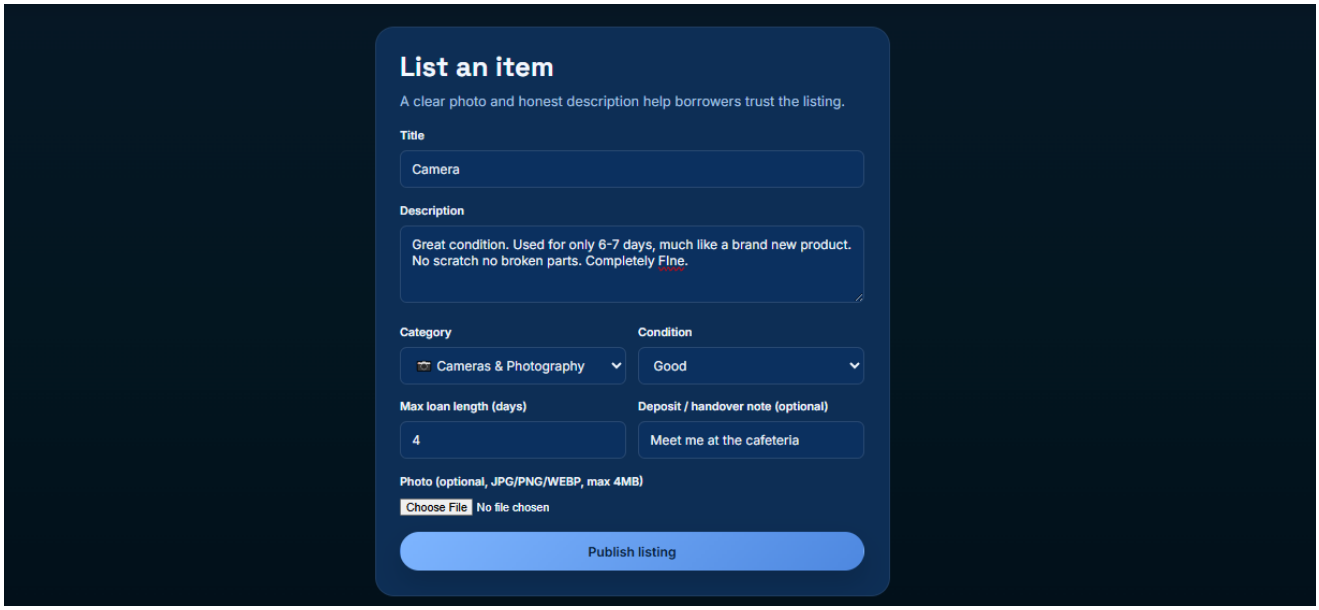


Fig 4 : Here's how the user will list an item. While listing the item, the user must have to provide necessary information about the product and keeping in mind with all those things, I have kept the following options – description, category, condition, maximum loan length, where to meet and handover the product at and an optional button to simply to put a picture of the listed item.

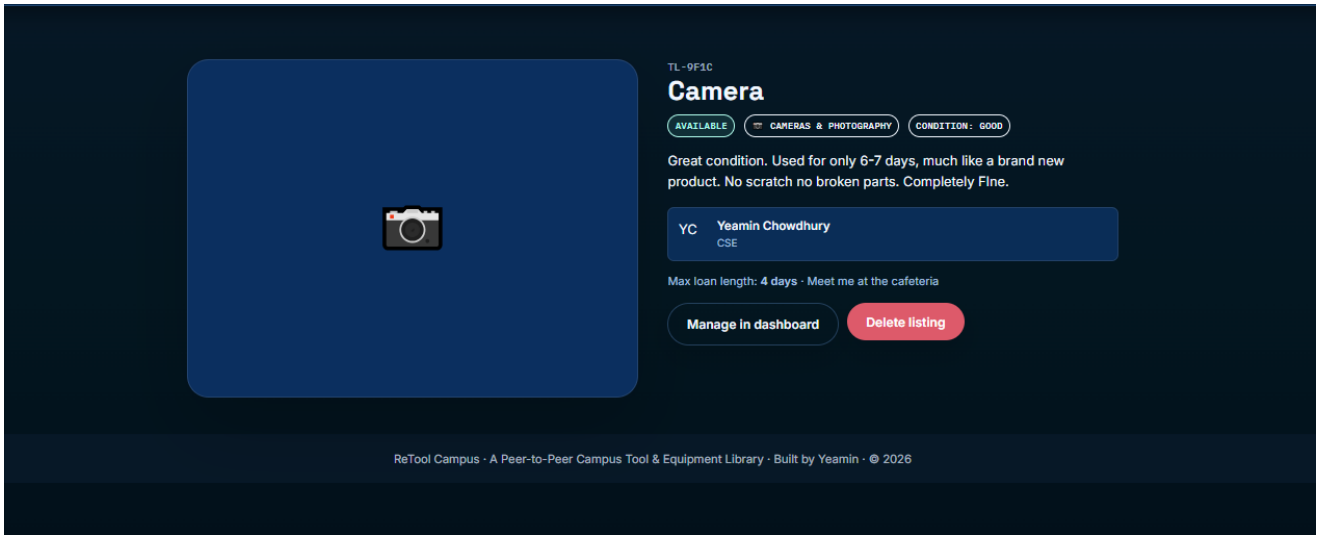


Fig 5 : After listing, this interface will appear confirming the listing of my product has been successfully executed

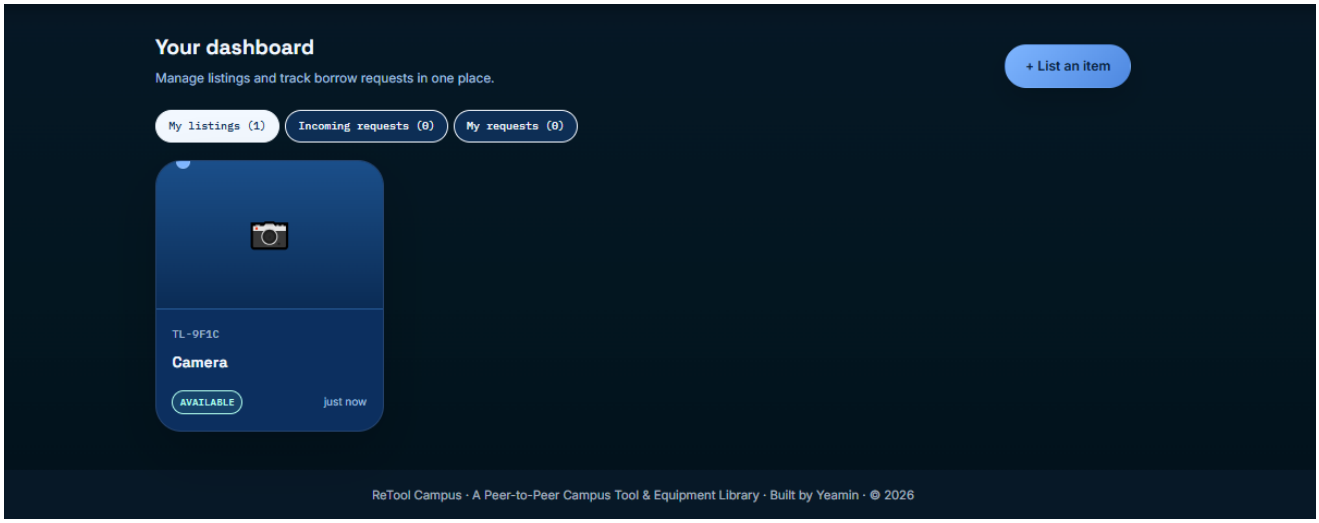
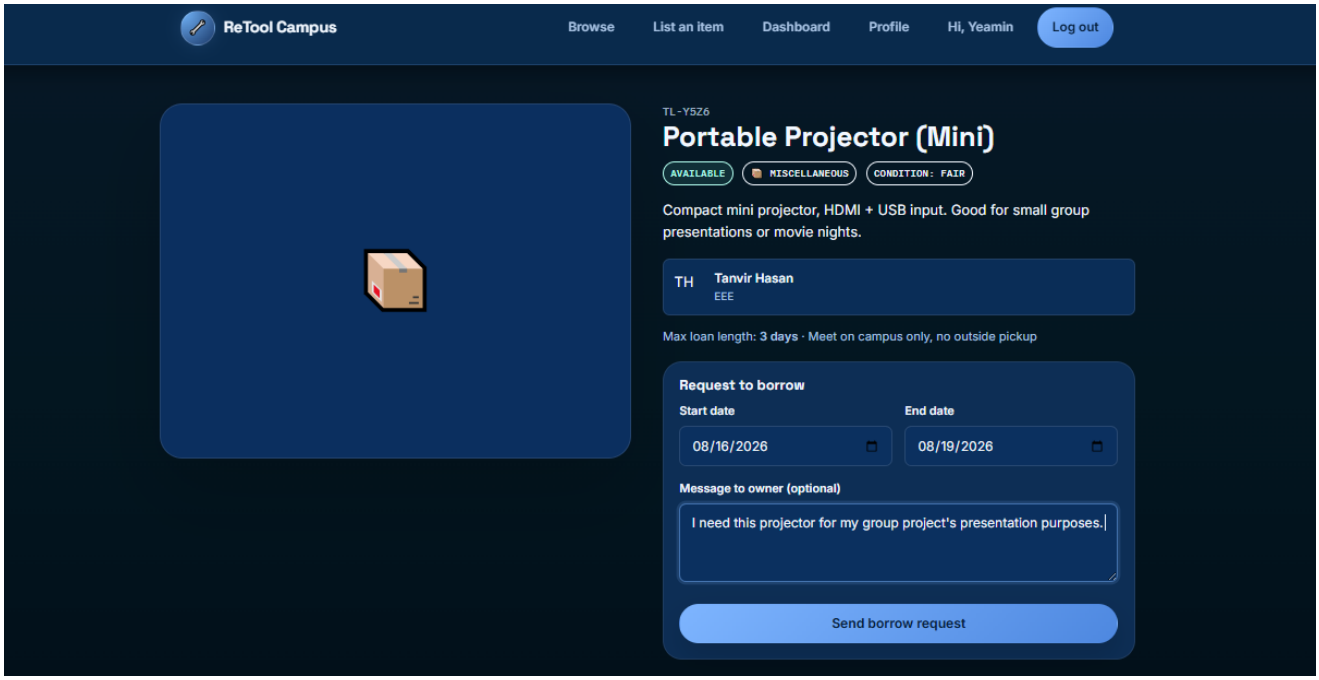
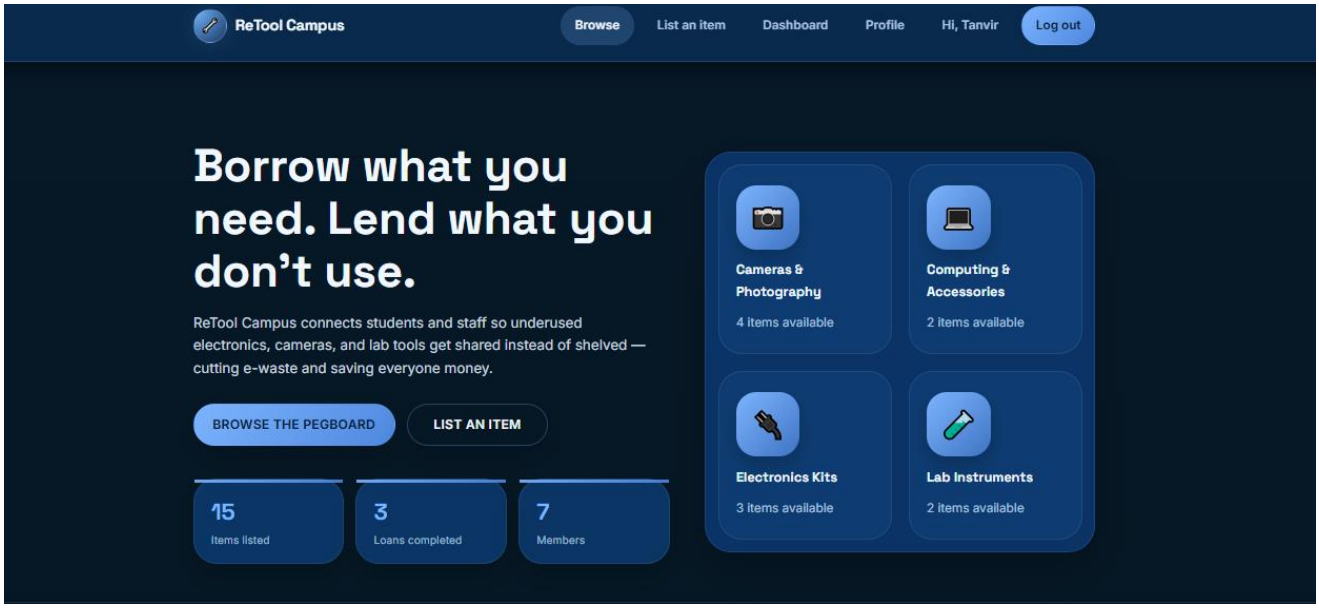


Fig 6 : The user dashboard will now display the items that has been listed


ReTool Campus
Browse
List an item
Dashboard
Profile
Hi, Yeamin
Log out
TL-Y5Z6
Portable Projector (Mini)
AVAILABLE
MISCELLANEOUS
CONDITION: FAIR
Compact mini projector, HDMI + USB input. Good for small group presentations or movie nights.
TH
Tanvir Hasan
EEE
Max loan length: 3 days · Meet on campus only, no outside pickup
Request to borrow
Start date
End date
08/16/2026
08/19/2026
Message to owner (optional)
I need this projector for my group project's presentation purposes.
Send borrow request


ReTool Campus
Browse
List an item
Dashboard
Profile
Hi, Tanvir
Log out
Borrow what you need. Lend what you don't use.
ReTool Campus connects students and staff so underused electronics, cameras, and lab tools get shared instead of shelved — cutting e-waste and saving everyone money.
BROWSE THE PEGBOARD
LIST AN ITEM
15
Items listed
3
Loans completed
7
Members
Cameras & Photography
4 items available
Computing & Accessories
2 items available
Electronics Kits
3 items available
Lab Instruments
2 items available

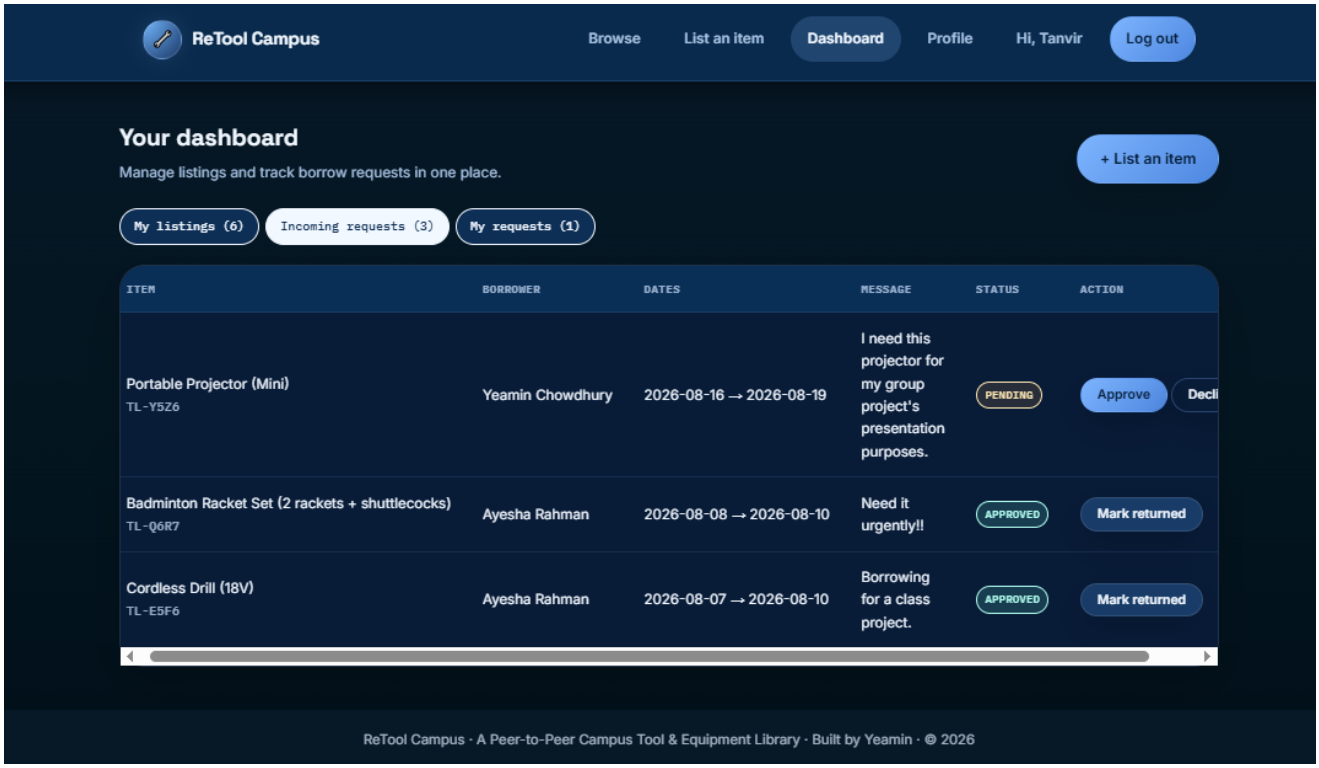


Fig 7 : Submitting, approving, declining, and cancelling borrow requests, and marking a completed loan as returned. Here I(Yeamin) had sent a borrow request to tanvir (another user), he can now see the incoming borrow requests from me and have the option to approve or decline my request.

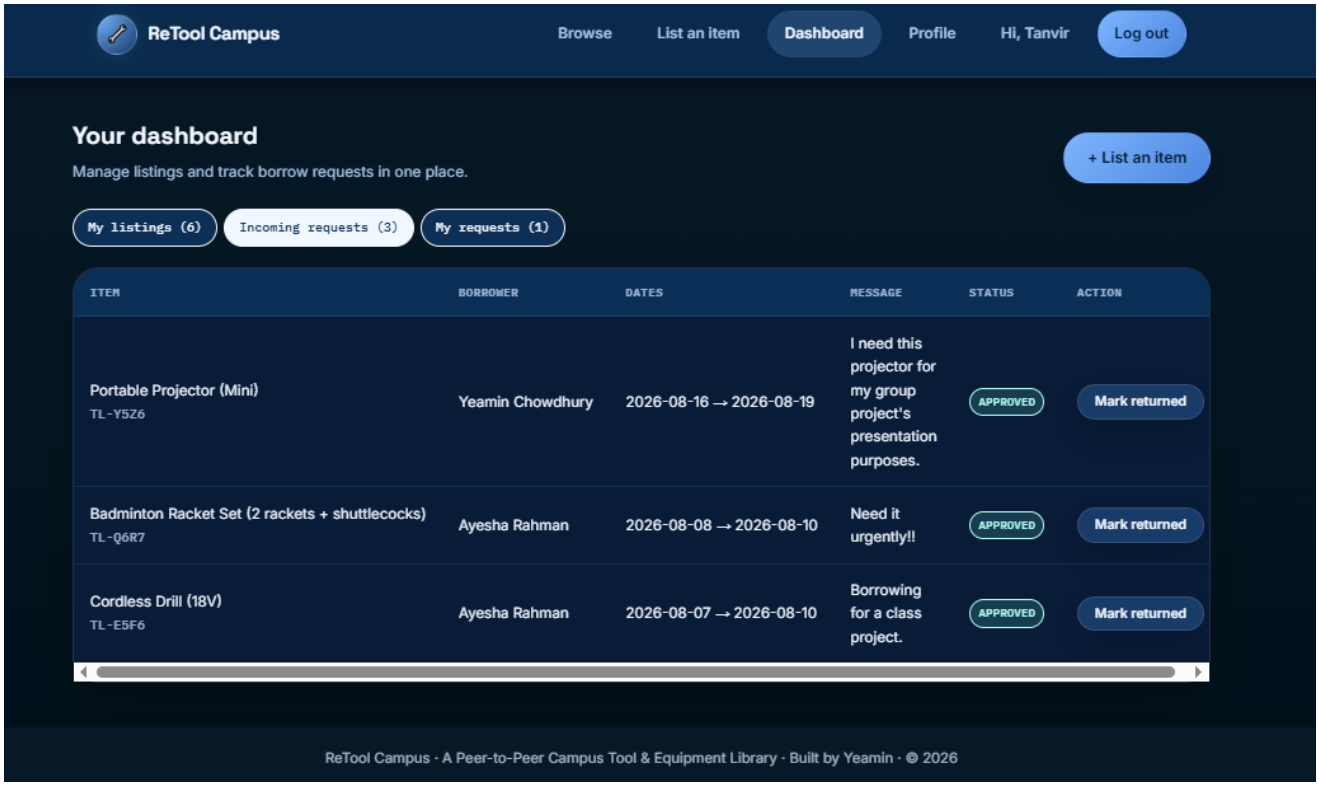


Fig 8 : Dashboard tabs correctly separating a user's own listings from incoming and outgoing borrow requests.

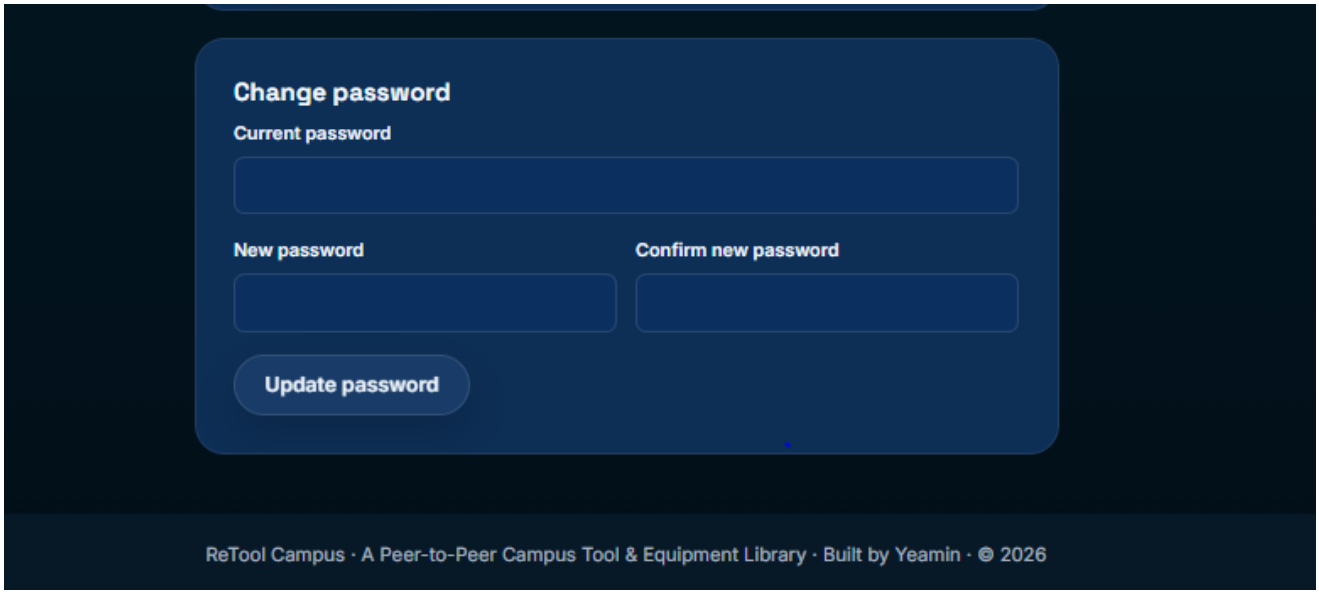


Fig 9 : Profile updates and password changes, with re-verification of the current password.

Ethical Framework

Beyond functional correctness, the lab requires the frontend to be evaluated against a recognized accessibility standard. ReTool Campus is assessed here against the Web Content Accessibility Guidelines (WCAG) 2.0, published by the W3C, which organizes accessibility requirements under four principles — the interface must be Perceivable, Operable, Understandable, and Robust (POUR) — each broken down into testable success criteria at Levels A, AA, and AAA. The mapping below reviews a representative set of Level A and AA success criteria most relevant to a data-driven, form-heavy application like this one, and records whether the current implementation addresses each criterion, based directly on the PHP/HTML markup used across index.php, item.php, add_item.php, register.php, login.php, dashboard.php, and profile.php.

Ethical & Technical Compliance Mapping Form

Framework: WCAG 2.0

Project Title: ReTool Campus— Peer-to-Peer Campus Tool & Equipment Library
Student: Yeamin Eram Chowdhury (ID: 242014209)
Course: CSE 3120 — Web Programming

SC	WCAG 2.0 Success Criterion	Applicability Status	Frontend Practice / Evidence in Retool Campus
1.1.1	Non-text Content (A)	Partially Addressed	Category icons render as visible emoji text (e($category_icon)) so they already have a text form, but uploaded item photos are rendered as CSS background-image (item-photo / detail-photo divs in index.php, item.php, dashboard.php) rather than <img> with an alt attribute — screen-reader users get no description of an uploaded photo.
1.3.1	Info and Relationships (A)	Addressed	Every form field uses an explicit, programmatically linked <label for="..."> paired with a matching input id (register.php, login.php, add_item.php, item.php's borrow form, profile.php), so field purpose is conveyed through markup, not just visual layout.
1.4.3	Contrast (Minimum) (AA)	Addressed	The project's CSS design system (css/style.css) is documented as using high-contrast text/background color pairs, verified with a browser contrast checker per the project's own security/accessibility notes.
2.1.1	Keyboard (A)	Addressed	All interactive controls are native HTML elements — <a>, <button>, <input>, <select>, <textarea> — across every page (index.php filter bar, item.php borrow form, dashboard.php action buttons). No custom JavaScript widgets intercept or replace default keyboard behavior.
2.4.4	Link Purpose (In Context) (A)	Addressed	Item cards (index.php, dashboard.php) wrap the entire card in a single <a> whose visible content — asset code and title — describes the destination, so link purpose is clear from the link text and its immediate context.
2.4.6	Headings and Labels (AA)	Addressed	Pages use descriptive, hierarchical headings (e.g. "Your dashboard", "List an item", the item's own title as <h1> in item.php) instead of generic labels like "Page 1".
3.2.2	On Input (A)	Addressed	No form submits automatically on a change or focus event; the category/status filters on index.php and every other form require an explicit Filter/Publish/Send button click, so users are never surprised by an unrequested context change.
3.3.1	Error Identification (A)	Partially Addressed	Validation errors are surfaced in visible text blocks above each form (register.php, login.php, add_item.php,

SC	WCAG 2.0 Success Criterion	Applicability Status	Frontend Practice / Evidence in Retool Campus
			item.php, profile.php), but they are not individually associated with the offending field via aria-describedby, so screen-reader users must read the whole error list rather than jump to the specific field.
3.3.2	Labels or Instructions (A)	Addressed	Beyond labels, most inputs carry helper text or placeholders describing the expected format (e.g. password length hints in register.php and profile.php, placeholder examples in add_item.php).
4.1.2	Name, Role, Value (A)	Addressed	Because all controls are native HTML form elements rather than custom ARIA widgets, the browser and assistive technologies expose correct name, role, and value information automatically without extra ARIA authoring.

Summary and Next Steps

Of the ten success criteria reviewed, eight are fully addressed by the current markup because the interface relies almost entirely on native, semantic HTML elements (labels, buttons, links, headings) rather than custom widgets — a pattern that tends to yield strong accessibility "for free." Two criteria are only partially addressed: (1) uploaded item photos lack a text alternative because they are rendered as CSS background images rather than <img alt="..."> elements, and (2) form validation errors are grouped in a single alert block instead of being linked to their specific field with aria-describedby. Both are realistic, scoped fixes for a future iteration: switching photo rendering to an <img> tag with a generated alt attribute (e.g. the item title), and adding aria-describedby / aria-invalid attributes to individual fields when they fail validation, would raise the project's WCAG 2.0 conformance without any structural redesign.

Conclusion

ReTool Campus successfully demonstrates a complete, database-driven web application built with plain PHP and MySQL, covering the full lifecycle of a peer-to-peer lending platform: account creation and authentication, item listing, search and discovery, a multi-stage borrow-request workflow, and account/profile management. The relational schema is normalized and referentially consistent across five interlinked tables, and every state-changing operation in the codebase is protected by parameterized queries, CSRF tokens, output escaping, and explicit ownership checks — directly satisfying Task 1 through Task 3 of the lab's requirements.

The accessibility review conducted against WCAG 2.0 shows that the project's reliance on native, semantic HTML elements gives it a solid accessibility baseline out of the box, while also surfacing two concrete, addressable gaps around image alternatives and field-level error association. Taken together, the implementation, results, and ethical framework sections show a working prototype that not only fulfils the assigned functionality but also reflects an awareness of the security and accessibility responsibilities that come with building a real, multi-user web application.

Future work could extend the platform with email notifications on request status changes, a star-rating/review system after completed loans, an administrative moderation panel, and automated overdue-loan reminders — all noted as natural extensions in the project's own README — alongside closing the two partially-addressed WCAG gaps identified above.